

Recursion DSA Handbook

A Complete, Practical Guide to Recursion Patterns in Python

PART 1: RECURSION FUNDAMENTALS

- 1.1 What is Recursion?
- 1.2 Core Terminology
- 1.3 Types of Recursion
- 1.4 Recursion vs Iteration
- 1.5 How to Read This Handbook

PART 2: PATTERNS

- 2.1 Linear Recursion
- 2.2 Tree / Multiple Recursion
- 2.3 Recursion on Strings & Arrays
- 2.4 Divide and Conquer Recursion
- 2.5 Recursion on Linked Lists
- 2.6 Recursion on Trees
- 2.7 Recursion with Memoization (Bridge to DP)
- 2.8 Tail Recursion & Mutual/Indirect Recursion

PART 3: QUICK REFERENCE CHEAT SHEET

Decision Guide — “How Do I Set Up the Recursion?”

PART 1: FUNDAMENTALS

1.1 What is Recursion?

Recursion is when a function calls itself to solve a smaller instance of the same problem, until it reaches a case simple enough to answer directly (the base case). Every recursive call adds a new frame to the call stack; when a call returns, control (and its local variables) pop back to the caller. Recursion is really just induction expressed as code: assume the function already works correctly for a smaller input, and use that to build the answer for the current input.

1.2 Core Terminology

Base Case	The simplest input(s) for which the function returns an answer directly, without recursing. Every recursive function needs at least one.
Recursive Case	The general case where the function calls itself on a smaller/simpler input and combines that result to answer the current input.
Call Stack	The stack of function frames maintained by the runtime; each recursive call pushes a frame, each return pops one.
Stack Frame	The memory block holding one call's parameters, local variables, and return address.
Recursion Tree	A tree showing every recursive call made, useful for visualizing time complexity and repeated subproblems.
Recursion Depth	The maximum number of nested calls alive at once — determines stack space used and risk of stack overflow.
Induction Hypothesis (IH)	The assumption that the recursive call already correctly solves the smaller subproblem — the mental trick for writing/verifying recursion.
Tail Call	A recursive call that is the very last operation in a function, with nothing left to do after it returns.
Backtracking	Recursion that tries a choice, recurses, then undoes ("un-chooses") the choice to try alternatives — covered in its own handbook.

1.3 Types of Recursion

By Number of Recursive Calls

- **Linear Recursion** — each call makes exactly one recursive call (factorial, sum of array). Recursion tree is a straight line.
- **Tree / Multiple Recursion** — each call makes 2+ recursive calls (Fibonacci, subsets, Tower of Hanoi). Recursion tree branches, often causing exponential blowup without memoization.

By Structure

- **Direct Recursion** — a function calls itself directly.
- **Indirect / Mutual Recursion** — function A calls function B, which calls function A again.

- **Tail Recursion** — the recursive call is the last action; some languages optimize this into a loop (Python does NOT optimize this — important caveat).
- **Non-Tail (Head) Recursion** — work happens AFTER the recursive call returns (e.g., printing after the call) — this is what lets recursion do things like reverse a string or unwind a stack.

1.4 Recursion vs Iteration

Readability	Recursion often mirrors the problem's natural definition (trees, divide-and-conquer); iteration can be more verbose for the same logic.
Memory	Recursion uses $O(\text{depth})$ call-stack space; iteration typically uses $O(1)$ extra space unless you maintain an explicit stack/queue.
Speed	Iteration is usually faster in Python since function-call overhead is relatively high and Python has no tail-call optimization.
Stack Limits	Python's default recursion limit is ~1000 frames (<code>sys.setrecursionlimit</code> can raise it, but a true stack overflow risk remains for very deep inputs).
When to prefer recursion	Tree/graph traversal, divide-and-conquer, backtracking, problems with a naturally self-similar/inductive definition.
When to prefer iteration	Simple linear scans, very large inputs where stack depth is a concern, performance-critical inner loops.

1.5 How to Read This Handbook

For every pattern below you will get, in this exact order: (1) What it's for, (2) Use Cases, (3) Limitations, (4) Time & Space Complexity, (5) Approach (short), (6) Keywords & Constraints that hint you should use this pattern, (7) Fully commented Python code, and (8) Practice Questions with a short reasoning of why the pattern fits.

PART 2: PATTERNS & ALGORITHMS

2.1 Linear Recursion

What it's for: The simplest recursive shape: each call makes exactly one recursive call on a strictly smaller input, then combines it with local work to produce the answer. This is the on-ramp for all recursive thinking.

Use Cases:

- Computing factorial, power, sum/product of a range.
- Summing or finding max/min of an array or list.
- Counting digits, reversing a number, digit sum.
- Any "do X for n, in terms of X for n-1" definition.

Limitations:

- Still uses $O(n)$ stack space — an iterative loop does the same work in $O(1)$ space.
- No branching means no exponential blowup, but very large n can still hit Python's recursion limit.
- Doesn't by itself avoid repeated work — that only becomes an issue once calls branch (see 2.2).

Time Complexity: $O(n)$ — one call per unit of input reduction **Space Complexity:** $O(n)$ — call stack depth equals input size

Approach (short): Identify the smallest input you can answer immediately (base case). Express the answer for size n in terms of the answer for a smaller size (usually $n-1$), then combine.

Keywords & Constraints: "in terms of $n-1$ ", "compute for n given $n-1$ ", factorial/sum/product phrasing; constraints $n \leq 10^4$ - 10^5 before stack depth becomes a concern.

```
def factorial(n):
    """Base case: 0! = 1. Recursive case: n! = n * (n-1)!"""
    if n == 0:
        return 1
    return n * factorial(n - 1)      # base case stops the recursion
                                    # trust factorial(n-1) is correct (IH), multiply by n

def array_sum(arr, i=0):
    """Sum of arr[i:] expressed via sum of arr[i+1:]."""
    if i == len(arr):
        return 0
    return arr[i] + array_sum(arr, i + 1)

def count_digits(n):
    """Number of digits in n, via digits of n // 10."""
    if n < 10:
        return 1
    return 1 + count_digits(n // 10)
```

Practice Questions — Linear Recursion

Q1. Factorial of N Compute $n!$ recursively. *Approach:* Direct linear recursion: $n! = n * (n-1)!$, base case $0! = 1$.

Q2. Sum of Natural Numbers Sum 1 to n without loops. *Approach:* $\text{sum}(n) = n + \text{sum}(n-1)$, base case $\text{sum}(0) = 0$.

Q3. Power of a Number (Naive) Compute x^n recursively. *Approach:* $\text{pow}(x, n) = x * \text{pow}(x, n-1)$; base case $\text{pow}(x, 0) = 1$ — later improved by divide-and-conquer fast power.

Q4. Sum of Digits Sum the digits of a number. *Approach:* $\text{digitSum}(n) = n \% 10 + \text{digitSum}(n // 10)$; base case $n < 10$ returns n .

Q5. Check if Array is Sorted Recursively verify array is non-decreasing. *Approach:* Compare `arr[0]` and `arr[1]`, recurse on `arr[1:]` — false as soon as one adjacent pair violates order.

Q6. Recursive Maximum in Array Find the max element without loops. *Approach:* `max(arr) = max(arr[0], max(arr[1:]))`; base case single-element array returns that element.

2.2 Tree / Multiple Recursion

What it's for: Each call branches into two or more recursive calls, forming a recursion tree rather than a line. This is the natural shape for "include or exclude", "go left or right", and combinatorial problems.

Use Cases:

- Fibonacci-style sequences defined by 2+ prior terms.
- Generating all subsets via an include/exclude decision at each index.
- Tower of Hanoi — move n disks by recursing on $n-1$ twice.
- Any "for each choice, branch and recurse" combinatorial structure (precursor to full backtracking).

Limitations:

- Without memoization, branching recursion is frequently exponential (e.g., naive Fibonacci is $O(2^n)$) because the same subproblem is recomputed many times.
- Recursion tree can be very wide, making it hard to reason about without drawing it out.
- Easy to accidentally explore the same state twice — watch for overlapping subproblems (signals you may want 2.7 Memoization).

Time Complexity: $O(\text{branches}^{\text{depth}})$ in the naive case — e.g., $O(2^n)$ for plain Fibonacci or subset generation

Space Complexity: $O(\text{depth})$ for the call stack (NOT $O(\text{number of calls})$ — only the active path is on the stack at once)

Approach (short): At each call, make 2+ recursive calls representing distinct choices/subproblems, then combine their results (sum, compare, concatenate). Draw the recursion tree for small n to see repeated work.

Keywords & Constraints: "two previous terms", "include or exclude", "all subsets", "number of ways", small n ($\leq 30-35$) hints exponential recursion is acceptable without memoization.

```
def fibonacci(n):
    """Branches into 2 calls each time -- exponential without memoization."""
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)    # base cases: fib(0)=0, fib(1)=1
                                                # combine two smaller subproblems

def subsets(nums, i=0, current=None):
    """Generate all subsets via include/exclude branching at each index."""
    if current is None:
        current = []
    if i == len(nums):
        print(current)                            # base case: made a decision for every index
        return
    subsets(nums, i + 1, current)                  # branch 1: exclude nums[i]
    subsets(nums, i + 1, current + [nums[i]])      # branch 2: include nums[i]

def tower_of_hanoi(n, source, aux, target):
    """Move n disks from source to target using aux, via 2 recursive calls."""
    if n == 0:
        return
    tower_of_hanoi(n - 1, source, target, aux)    # move top n-1 disks out of the way
    print(f"Move disk {n} from {source} to {target}")
    tower_of_hanoi(n - 1, aux, source, target)    # move those n-1 disks onto target
```

Practice Questions — Tree / Multiple Recursion

Q1. Nth Fibonacci Number (Naive) Compute $\text{fib}(n)$ with plain recursion. *Approach:* Two-way branch $\text{fib}(n-1)+\text{fib}(n-2)$; illustrates why overlapping subproblems demand memoization for large n .

Q2. Tower of Hanoi Move n disks between 3 pegs following the rules; count/print moves. *Approach:* Classic 2-call recursion: move $n-1$ disks out of the way, move disk n , move the $n-1$ disks back on top.

Q3. Generate All Subsets (Power Set) Print every subset of an array. *Approach:* Branch at each index into "exclude" and "include" — 2^n leaves in the recursion tree, one per subset.

Q4. Count Ways to Climb Stairs Count ways to reach step n taking 1 or 2 steps at a time. *Approach:* $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$, a Fibonacci-shaped branching recursion — naturally leads to DP once n grows.

Q5. Print All Binary Strings of Length N Generate every string of 0s and 1s of length n . *Approach:* At each position branch into placing '0' or '1', recurse on position+1 — 2^n total strings, one per full-depth path.

Q6. Grid Ways (Robot Paths, Unweighted) Count paths from top-left to bottom-right moving only right or down. *Approach:* $\text{ways}(r, c) = \text{ways}(r-1, c) + \text{ways}(r, c-1)$; overlapping subproblems here are the classic motivation for memoized/DP grid paths.

2.3 Recursion on Strings & Arrays

What it's for: Using recursion to process sequences index-by-index or by shrinking from both ends, which naturally expresses problems like reversal, palindrome checking, and pattern matching.

Use Cases:

- Reversing a string or array in place using two-pointer recursion.
- Palindrome checking by comparing ends and recursing inward.
- Basic pattern/substring matching and simple string transformations.
- Building up or tearing down a string one character at a time (e.g., removing duplicates).

Limitations:

- Python strings are immutable, so recursive string building via concatenation is $O(n)$ per call, making some naive solutions $O(n^2)$ overall — prefer list accumulation where possible.
- For very long strings/arrays, recursion depth can exceed Python's limit; an iterative two-pointer loop is usually preferable in production code.
- Easy to off-by-one on the "meet in the middle" base case (single element vs. empty range).

Time Complexity: $O(n)$ for single-pass patterns; $O(n^2)$ if concatenation happens at every level **Space Complexity:** $O(n)$ recursion depth (plus $O(n)$ if building new strings at each level)

Approach (short): Shrink the problem from one end (index i onward) or from both ends (i and j moving toward each other) each call; combine the current character's contribution with the recursive result on the rest.

Keywords & Constraints: "reverse", "palindrome", "two pointers", "in terms of substring", constraints $n \leq 10^4$ - 10^5 .

```
def reverse_string(s, i=0, j=None):
    """Two-pointer recursion: swap ends, recurse inward. Works on a mutable list of chars."""
    if j is None:
        j = len(s) - 1
    if i >= j:
        return s
    s[i], s[j] = s[j], s[i]
    return reverse_string(s, i + 1, j - 1)
```

```
def is_palindrome(s, i=0, j=None):
    """Compare outer characters, shrink inward -- false as soon as a mismatch is found."""
    if j is None:
        j = len(s) - 1
    if i >= j:
        return True
    if s[i] != s[j]:
        return False
    return is_palindrome(s, i + 1, j - 1)
```

```
def remove_char(s, ch, i=0):
    """Build a new string skipping all occurrences of ch."""
    if i == len(s):
        return ""
    rest = remove_char(s, ch, i + 1)
    return rest if s[i] == ch else s[i] + rest
```

Practice Questions — Recursion on Strings & Arrays

Q1. Reverse a String Reverse a string/array in place. *Approach:* Two-pointer recursion swapping $s[i]$ and $s[j]$, recursing on $(i+1, j-1)$ until pointers meet or cross.

Q2. Check Palindrome Determine whether a string reads the same forwards and backwards. *Approach:* Compare first and last characters, recurse inward; return False immediately on any mismatch.

Q3. Remove All Occurrences of a Character Delete every instance of a target character from a string. *Approach:* Recurse to the end first, then at each level decide to include or skip the current character while building the result.

Q4. Check if String Contains Only Digits Validate every character is 0-9. *Approach:* Check `s[0]` is a digit, then recurse on `s[1:]` — base case empty string returns `True`.

Q5. String to Integer (atoi, simplified) Convert a numeric string to an integer recursively. *Approach:* $\text{value}(s) = \text{value}(s[:-1]) * 10 + \text{int}(s[-1])$; base case single character returns `int(s)`.

Q6. Find All Occurrences of a Substring (Naive) Return starting indices where a pattern occurs in text. *Approach:* Check the pattern at index `i`, recurse on `i+1` for the rest — straightforward index-advance recursion (compare with KMP for the optimized non-recursive version).

2.4 Divide and Conquer Recursion

What it's for: Splits a problem into independent smaller subproblems (usually roughly half the size), solves each recursively, then combines their results. The classic template behind fast sorting, searching, and exponentiation.

Use Cases:

- Sorting: Merge Sort, Quick Sort.
- Searching: Binary Search and its variants.
- Fast exponentiation (x^n in $O(\log n)$).
- Closest pair of points, maximum subarray (Kadane's divide-and-conquer variant), matrix multiplication (Strassen's).

Limitations:

- Only pays off when subproblems are truly independent — if they overlap, plain divide-and-conquer redoes work (that's when DP/memoization is the better fit).
- The "combine" step can dominate cost if not done carefully (e.g., naive merge in merge sort still needs $O(n)$ extra space).
- Quick Sort's divide step is not balanced in the worst case, degrading to $O(n^2)$ without a good pivot strategy.

Time Complexity: Typically $O(n \log n)$ when the problem halves each time and combine is $O(n)$; varies by algorithm (see Master Theorem) **Space Complexity:** $O(\log n)$ recursion depth for balanced splits; extra $O(n)$ if merging into new arrays

Approach (short): Split input into (usually 2) smaller, independent pieces → recursively solve each → combine/merge the sub-results into the answer for the full input.

Keywords & Constraints: "divide into halves", "merge", "sort", "search in $\log n$ ", "fast power", constraints $n \leq 10^5$ - 10^6 for $O(n \log n)$ solutions.

```
def merge_sort(arr):
    """Divide array in half, sort each half, merge sorted halves."""
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return _merge(left, right)

def _merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def binary_search(arr, target, lo=0, hi=None):
    """Halve the search space each call --  $O(\log n)$ ."""
    if hi is None:
        hi = len(arr) - 1
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if arr[mid] == target:
        return mid
    if arr[mid] < target:
        return binary_search(arr, target, mid + 1, hi)
    return binary_search(arr, target, lo, mid - 1)
```

```
def fast_power(x, n):
    """Compute  $x^n$  in  $O(\log n)$  by squaring instead of  $n$  multiplications."""
    if n == 0:
        return 1
    half = fast_power(x, n // 2)
    return half * half if n % 2 == 0 else half * half * x
```

Practice Questions — Divide and Conquer Recursion

Q1. Merge Sort Sort an array using divide and conquer. *Approach:* Split into halves, recursively sort each, merge two sorted halves in $O(n)$ — classic $O(n \log n)$ template.

Q2. Quick Sort Sort an array by partitioning around a pivot. *Approach:* Partition so smaller elements land left of pivot and larger land right, recurse on both sides — unlike merge sort, the "combine" step is free but the "divide" step does the work.

Q3. Binary Search Find target index in a sorted array. *Approach:* Compare target to the middle element, recurse into only the half that could contain it — halves the search space every call.

Q4. Pow(x, n) — Fast Exponentiation Compute x raised to n efficiently. *Approach:* Recurse on $x^{(n/2)}$ and square it instead of multiplying x by itself n times — turns $O(n)$ into $O(\log n)$.

Q5. Maximum Subarray Sum (Divide and Conquer) Find the max sum contiguous subarray. *Approach:* Best subarray is entirely in left half, entirely in right half, or crosses the midpoint — recurse on both halves and compute the crossing sum in $O(n)$ to combine.

Q6. Count Inversions in an Array Count pairs (i, j) where $i < j$ but $arr[i] > arr[j]$. *Approach:* Piggyback on merge sort's merge step — whenever an element from the right half is placed before remaining left-half elements, that's a batch of inversions counted for free.

2.5 Recursion on Linked Lists

What it's for: Recursion is a natural fit for linked lists because a list is itself a recursive structure: a node plus "the rest of the list", which is itself a (smaller) linked list.

Use Cases:

- Reversing a linked list, or reversing in groups of k.
- Finding the middle, detecting cycles conceptually (though iterative Floyd's is preferred for cycles).
- Merging two sorted linked lists.
- Deleting nodes, removing duplicates, flattening nested/multilevel lists.

Limitations:

- Recursion depth equals list length — for very long lists ($>10^4$ in Python) this risks hitting the recursion limit; iterative pointer manipulation is safer for production code.
- Recursive solutions can accidentally hold references to the whole list on the stack, using $O(n)$ auxiliary space versus $O(1)$ for a well-written iterative version.
- Head-recursive list algorithms (acting after the recursive call returns) are easy to write incorrectly regarding which node's `.next` gets updated.

Time Complexity: $O(n)$ — one call per node **Space Complexity:** $O(n)$ call stack (this is the main downside versus the $O(1)$ -space iterative versions)

Approach (short): Treat "head.next onward" as a smaller linked list; recurse on it first (trusting the IH that it's correctly processed), then fix up pointers involving the current head on the way back up.

Keywords & Constraints: "reverse linked list", "merge two lists", "kth node", "remove node", list length not given as huge ($\leq 10^4$ for recursive solutions).

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

def reverse_list(head):
    """Recursively reverse a singly linked list."""
    if head is None or head.next is None:
        return head
    new_head = reverse_list(head.next)
    head.next.next = head
    head.next = None
    return new_head
    # base case: 0 or 1 node is already reversed
    # IH: rest of the list is now reversed, tail is head.next
    # make the (old) next node point back to current
    # current node becomes the new tail
    # new_head bubbles up unchanged from the base case

def merge_two_sorted(l1, l2):
    """Merge two sorted linked lists recursively."""
    if l1 is None:
        return l2
    if l2 is None:
        return l1
    if l1.val <= l2.val:
        l1.next = merge_two_sorted(l1.next, l2)
        return l1
    l2.next = merge_two_sorted(l1, l2.next)
    return l2
    # base case: one list exhausted
    # IH: rest is merged correctly

def get_length(head):
    if head is None:
        return 0
    return 1 + get_length(head.next)
```

Practice Questions — Recursion on Linked Lists

Q1. Reverse a Linked List Reverse a singly linked list. *Approach:* Recurse to the end first, then flip each next pointer on the way back up — classic head-recursion.

Q2. Merge Two Sorted Linked Lists Merge two sorted lists into one sorted list. *Approach:* Compare heads, attach the smaller to the recursively merged remainder of both lists.

Q3. Remove Duplicates from Sorted Linked List Delete consecutive duplicate values. *Approach:* Compare current node to the next; skip the next node if equal, otherwise recurse normally — duplicates in a sorted list are always adjacent.

Q4. Find the Middle Node Return the middle node of a linked list. *Approach:* Recursive length-count first (or fast/slow pointer iteratively) then recurse that many steps — shown here to contrast with the simpler iterative two-pointer approach.

Q5. Reverse Linked List in Groups of K Reverse every k nodes as a group. *Approach:* Reverse the first k nodes iteratively/recursively, then recursively process the remainder of the list and attach it to the tail of the reversed group.

Q6. Check if Linked List is a Palindrome Determine whether the list reads the same forwards and backwards. *Approach:* Recurse to the end to get a "back pointer" unwinding while comparing against a forward pointer maintained via a nonlocal/outer variable — clever use of the call stack as an implicit reversed traversal.

2.6 Recursion on Trees

What it's for: Trees are inherently recursive (a tree is a node plus subtrees, which are themselves trees), making recursion the default and most natural tool for nearly all tree problems.

Use Cases:

- Traversals: preorder, inorder, postorder, and their many variants.
- Computing height, depth, diameter, or balance of a tree.
- Path-sum problems, lowest common ancestor (LCA), subtree checks.
- Validating BST properties, serializing/deserializing trees.

Limitations:

- Recursion depth equals tree height — for a heavily skewed (linked-list-like) tree, this can hit Python's recursion limit even for a modest number of nodes.
- Problems needing level-by-level results (e.g., level order traversal) are usually better solved with iterative BFS rather than recursion.
- Passing large mutable accumulators (e.g., building a big path list) through recursive calls requires care to avoid state leaking between branches.

Time Complexity: $O(n)$ — each node visited once, for most traversal-style problems **Space Complexity:** $O(h)$ call stack where h is tree height — $O(\log n)$ for balanced trees, $O(n)$ worst case for skewed trees

Approach (short): Define the answer for a node in terms of the answers for its left and right subtrees (trusting the IH that recursive calls already solve those correctly), then combine with the current node's value.

Keywords & Constraints: "binary tree", "subtree", "path sum", "diameter", "LCA", "height/depth", $n \leq 10^4$ - 10^5 nodes.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def height(node):
    """Height of an empty tree is -1 (or 0 nodes = -1 edges); leaf has height 0."""
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))    # combine subtree heights

def inorder(node, result=None):
    if result is None:
        result = []
    if node is None:
        return result
    inorder(node.left, result)
    result.append(node.val)
    inorder(node.right, result)
    return result

def has_path_sum(node, target):
    """Does any root-to-leaf path sum exactly to target?"""
    if node is None:
        return False
    if node.left is None and node.right is None:    # leaf: check remaining target directly
        return node.val == target
    remaining = target - node.val
    return has_path_sum(node.left, remaining) or has_path_sum(node.right, remaining)

def diameter(node, ans=None):
    """Longest path between any two nodes (in edges), tracked via a mutable holder."""
```

```

if ans is None:
    ans = [0]
def depth(n):
    if n is None:
        return 0
    left_d = depth(n.left)
    right_d = depth(n.right)
    ans[0] = max(ans[0], left_d + right_d) # path through n = left depth + right depth
    return 1 + max(left_d, right_d)
depth(node)
return ans[0]

```

Practice Questions — Recursion on Trees

Q1. Preorder / Inorder / Postorder Traversal Traverse a binary tree in the 3 classic orders. *Approach:* Visit node before/between/after recursing on left and right subtrees respectively — same recursive skeleton, different placement of the "visit" step.

Q2. Height / Maximum Depth of Binary Tree Find the height of a binary tree. *Approach:* $\text{height}(\text{node}) = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$; base case null node has height -1 (or 0 depending on convention).

Q3. Diameter of Binary Tree Find the longest path between any two nodes. *Approach:* At every node, the best path through it is left-depth + right-depth; track a running max while computing depths bottom-up.

Q4. Lowest Common Ancestor (LCA) of a Binary Tree Find the LCA of two given nodes. *Approach:* Recurse into left and right; if both sides return non-null, current node is the LCA, otherwise propagate up the non-null side.

Q5. Path Sum (Root to Leaf) Check if any root-to-leaf path sums to a target. *Approach:* Subtract the current value from target and recurse into children; a leaf succeeds only if the remaining target exactly matches its value.

Q6. Validate Binary Search Tree Check if a binary tree satisfies BST ordering. *Approach:* Recurse while carrying a valid (low, high) range for each node's value — tightening the range as you descend left or right, rather than only comparing to immediate children.

2.7 Recursion with Memoization (Bridge to DP)

What it's for: Adds a cache to branching recursion so that each distinct subproblem is computed only once. This is the direct on-ramp from "plain recursion" to "top-down Dynamic Programming" — same recursive structure, but exponential blowup is eliminated by never recomputing a state twice.

Use Cases:

- Speeding up Fibonacci-style recurrences from exponential to linear time.
- Grid path counting / minimum path cost with overlapping subpaths.
- Any recursive solution whose recursion tree, when drawn, shows the same (arguments) appearing at multiple nodes.
- Prototyping DP solutions quickly before converting to iterative tabulation for extra speed.

Limitations:

- Only helps when subproblems actually overlap — pure divide-and-conquer on independent halves (merge sort) gets no benefit and shouldn't be memoized.
- Cache keys must be hashable; multi-dimensional state (e.g., (i, j, k)) needs a tuple key or nested dict/array.
- Still pays recursion's call-stack overhead per unique state — iterative tabulation (full DP handbook) is usually faster and avoids recursion-depth limits.

Time Complexity: $O(\text{number of distinct states} \times \text{work per state})$ — typically turns $O(2^n)$ into $O(n)$ or $O(n^2)$ etc.

Space Complexity: $O(\text{number of distinct states})$ for the cache, plus $O(\text{depth})$ call stack

Approach (short): Write the plain recursive solution first. Identify the arguments that define a "state". Before recursing, check a cache/dict for that state; if present return it, otherwise compute, store, then return.

Keywords & Constraints: "minimum/maximum number of ways", "overlapping subproblems", recursion tree with repeated arguments, n up to 10^3 - 10^4 where plain recursion would time out.

```
def fib_memo(n, cache=None):
    """Same recursive shape as plain Fibonacci, but  $O(n)$  instead of  $O(2^n)$ ."""
    if cache is None:
        cache = {}
    if n <= 1:
        return n
    if n in cache:
        return cache[n]
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]

from functools import lru_cache

@lru_cache(maxsize=None)
def grid_paths(r, c):
    """Count paths to (r, c) from (0,0) moving only right/down -- memoized via decorator."""
    if r == 0 or c == 0:
        return 1
    return grid_paths(r - 1, c) + grid_paths(r, c - 1)

def min_coin_count(coins, amount, cache=None):
    """Minimum coins to make `amount`; -1 if impossible. Manual dict-based memoization."""
    if cache is None:
        cache = {}
    if amount == 0:
        return 0
    if amount < 0:
        return float('inf')
    if amount in cache:
        return cache[amount]
    best = min((1 + min_coin_count(coins, amount - c, cache) for c in coins), default=float('inf'))
```



```
cache[amount] = best
return best
```

Practice Questions — Recursion with Memoization (Bridge to DP)

Q1. Nth Fibonacci Number (Memoized) Compute fib(n) efficiently for large n. *Approach:* Cache each fib(k) the first time it's computed — turns the exponential recursion tree into O(n) unique calls.

Q2. Grid Unique Paths Count paths from top-left to bottom-right of a grid. *Approach:* Memoize on (row, col) since the same cell is reached via many different routes in the plain recursive version.

Q3. Minimum Path Sum in a Grid Find the minimum cost path from top-left to bottom-right. *Approach:* $\text{minCost}(r, c) = \text{grid}[r][c] + \min(\text{minCost}(r-1, c), \text{minCost}(r, c-1))$; memoize on (r, c) to avoid recomputation.

Q4. Coin Change (Minimum Coins) Find fewest coins to make a target amount. *Approach:* Try every coin as the "last coin used", recurse on the remainder, and memoize on the remaining amount — without memoization this branches exponentially.

Q5. Longest Common Subsequence (Memoized Recursion) Find LCS length of two strings. *Approach:* Recurse on (i, j) indices into both strings with a match/no-match branch; memoize on the (i, j) pair since the same index combination recurs constantly.

Q6. Word Break Check if a string can be segmented into dictionary words. *Approach:* Try every valid prefix as the first word, recurse on the remaining suffix, and memoize on the starting index — the same suffix is reached via many different prefix splits.

2.8 Tail Recursion & Mutual/Indirect Recursion

What it's for: Two related refinements of the basic recursive template: tail recursion restructures a function so the recursive call is the very last operation (often via an accumulator), and mutual recursion is when two or more functions call each other to make progress together.

Use Cases:

- Rewriting linear recursion with an accumulator parameter to make the "pending work" explicit.
- Even/odd checks, alternating parsers, and state-machine-like logic split across cooperating functions.
- Recursive-descent parsers where grammar rules naturally call each other (expr calls term calls factor calls expr).
- Ping-pong style simulations (Hofstadter's female/male sequences, game turn alternation modeled recursively).

Limitations:

- Python's interpreter does NOT perform tail-call optimization — a "tail recursive" function in Python still consumes $O(n)$ stack space, unlike in languages such as Scheme or Haskell. Don't rely on it to avoid stack overflow in Python.
- Mutual recursion can be harder to trace/debug since control jumps between multiple functions.
- Both patterns can usually be flattened into a single iterative loop for better performance in Python — worth doing once correctness is established.

Time Complexity: $O(n)$ for both patterns in typical linear-recursive use cases **Space Complexity:** $O(n)$ call stack in Python regardless of tail position (no TCO); $O(1)$ if manually converted to a loop

Approach (short): Tail recursion: thread an accumulator through the parameters so each call carries the "answer so far" and the recursive call is the last statement, with no pending work after it returns. Mutual recursion: split the problem into cases handled by different functions that call one another.

Keywords & Constraints: "accumulator", "in terms of running total", parser/grammar rules referencing each other, alternating even/odd style problems.

```
def factorial_tail(n, acc=1):
    """Tail-recursive form: the recursive call is the LAST thing that happens.
    (Python still uses  $O(n)$  stack frames -- no TCO -- but this is the classic shape.)"""
    if n == 0:
        return acc
    return factorial_tail(n - 1, acc * n)    # nothing left to do after this call returns

def sum_tail(arr, i=0, acc=0):
    if i == len(arr):
        return acc
    return sum_tail(arr, i + 1, acc + arr[i]) # accumulator carries the running sum forward

def is_even(n):
    """Mutual recursion: is_even and is_odd define each other."""
    if n == 0:
        return True
    return is_odd(n - 1)

def is_odd(n):
    if n == 0:
        return False
    return is_even(n - 1)

def parse_expr(tokens, i):
    """Simplified recursive-descent sketch: expr calls term, term calls factor, factor may call expr back."""
    value, i = parse_term(tokens, i)
    while i < len(tokens) and tokens[i] in ('+', '-'):
        op = tokens[i]
        rhs, i = parse_term(tokens, i + 1)
```

```

        value = value + rhs if op == '+' else value - rhs
    return value, i

def parse_term(tokens, i):
    if tokens[i] == '(':
        value, i = parse_expr(tokens, i + 1)    # mutual call back up to parse_expr
        return value, i + 1                    # skip ')'
    return int(tokens[i]), i + 1

```

Practice Questions — Tail Recursion & Mutual/Indirect Recursion

Q1. Factorial — Tail Recursive Form Rewrite factorial using an accumulator. *Approach:* Thread $acc = acc * n$ through calls so the multiply happens BEFORE recursing, leaving nothing pending after the recursive call — the textbook tail-recursive shape (though Python still uses $O(n)$ stack frames).

Q2. Reverse a String — Tail Recursive Reverse a string using an accumulator instead of post-call work. *Approach:* Prepend/append characters into an accumulator argument on the way down instead of combining results on the way back up.

Q3. Even/Odd via Mutual Recursion Determine parity of n using two cooperating functions. *Approach:* $is_even(n)$ defers to $is_odd(n-1)$ and vice versa — classic minimal example of two functions making progress together.

Q4. Recursive Descent Expression Parser Evaluate a simple arithmetic expression with parentheses. *Approach:* $expr/term/factor$ functions call each other by grammar rule; a parenthesized sub-expression causes $factor$ to call back into $expr$ — genuine mutual recursion, not just two functions calling one another once.

Q5. Hofstadter Female and Male Sequences Compute $F(n)$ and $M(n)$ defined mutually in terms of each other. *Approach:* $F(n) = n - M(F(n-1))$ and $M(n) = n - F(M(n-1))$; a direct, if unusual, demonstration of two sequences defined purely via mutual recursion.

Q6. Ackermann Function Compute the famously fast-growing Ackermann function $A(m, n)$. *Approach:* Recurses on both m and n simultaneously (not strictly tail or mutual, but a good stress test) — illustrates how recursion depth/growth can explode even for tiny inputs, motivating why some recursions must be bounded carefully.

PART 3: QUICK REFERENCE CHEAT SHEET

Pattern	Purpose	Time	Space	Watch Out For
Linear Recursion	Reduce by 1 each call	$O(n)$	$O(n)$	No benefit over a simple loop besides style
Tree/Multiple Recursion	Branch into 2+ subcalls	$O(b^d)$ naive	$O(d)$	Exponential blowup without memoization
Strings/Arrays Recursion	Two-pointer / index-advance	$O(n)$	$O(n)$	Immutable string concatenation can add $O(n)$ per level
Divide and Conquer	Split, solve, combine	$O(n \log n)$ typical	$O(\log n)$	Only helps if subproblems are independent
Linked List Recursion	Head + rest-of-list	$O(n)$	$O(n)$	Iterative version is $O(1)$ space — use for huge lists
Tree Recursion	Node + subtrees	$O(n)$	$O(h)$	Skewed tree $\rightarrow O(n)$ depth, risk of stack limit
Memoized Recursion	Cache overlapping subproblems	$O(\text{states})$	$O(\text{states})$	Only helps if subproblems actually repeat
Tail/Mutual Recursion	Accumulator / cooperating fns	$O(n)$	$O(n)$ in Python	Python has NO tail-call optimization

Decision Guide — “Which Pattern Do I Use?”

1. Problem reduces cleanly to "same thing on $n-1$ "? $\rightarrow$ Linear Recursion
2. Each step naturally has 2+ choices/branches (include/exclude, move here or there)? $\rightarrow$ Tree/Multiple Recursion (add memoization if subproblems repeat)
3. Working with a string/array and comparing/shrinking from ends or index-by-index? $\rightarrow$ Recursion on Strings & Arrays
4. Problem splits into independent halves that combine cheaply (sort, search, exponentiation)? $\rightarrow$ Divide and Conquer
5. Structure IS a linked list (node + rest)? $\rightarrow$ Recursion on Linked Lists (but prefer iteration if list can be very long)
6. Structure IS a tree (node + subtrees)? $\rightarrow$ Recursion on Trees — the default and usually best tool
7. Drawing the recursion tree shows the same arguments appearing more than once? $\rightarrow$ Add Memoization (bridge straight into the DP handbook)
8. Need an explicit "running total" threaded through calls, or two rules that reference each other? $\rightarrow$ Tail Recursion / Mutual Recursion
9. Recursion needs to try a choice AND undo it to explore alternatives (not just branch and combine)? $\rightarrow$ That is Backtracking — see the dedicated Backtracking handbook